3D COMPUTER VISION

Report 3

Deep Learning on Point Clouds, Voxel Grids and Meshes

Reproducing the practical baselines of the tutorial *Point Clouds, Volumetric and Mesh-Based Deep Learning* (Section 7)

Student name	**Aggimalla Abhishek**
Registration no.	**23BDS004**
Course	**3D Computer Vision**
Datasets	**ModelNet40, ModelNet10, SHREC'16**
Models	**PointNet++ (SSG), 3D CNN on voxels, MeshCNN**
Tools	**Python 3.11, PyTorch, trimesh, Open3D, NumPy, SciPy, Matplotlib**
Submission date	**October 2026**

Abstract

A 3D shape can be given to a neural network as an unordered set of points, as a regular grid of occupied voxels, or as a mesh with explicit surface connectivity. Each representation calls for a different kind of network. This report reproduces the five practical exercises of the tutorial *Point Clouds, Volumetric and Mesh-Based Deep Learning* on a single Apple-silicon laptop. A synthetic point cloud validates the I/O path. PointNet++ is trained on ModelNet40 and reaches **88.1%** instance accuracy (82.7% class average) after 15 epochs. The tutorial's 3D CNN, after a bug in its voxelization code is fixed, is trained on all 4,899 ModelNet10 meshes at 32^3 and reaches **90.1%** (89.8% class average). MeshCNN reaches **85.0%** best test accuracy on the 30-class SHREC'16 benchmark and settles at 82.4% by epoch 100. Finally, one shape is converted between all representations, and the conversions are measured against the original surface with the Chamfer distance. Because the three classifiers use different datasets, the report compares them on cost and behaviour rather than ranking their accuracies against each other.

Keywords: point cloud, PointNet++, voxel grid, 3D convolution, mesh convolution, MeshCNN, ModelNet, SHREC, surface reconstruction.

Contents

1 Introduction . . . 4

1.1 Objective . . . 4

1.2 Contributions . . . 4

2 Background . . . 4

2.1 Point clouds: PointNet and PointNet++ . . . 4

2.2 Voxel grids: 3D convolution . . . 4

2.3 Meshes: MeshCNN . . . 5

3 Datasets . . . 5

4 Experimental Setup . . . 5

4.1 Hardware and software . . . 5

4.2 Training configuration . . . 6

5 Practical 7.1: Synthetic Point Cloud . . . 7

6 Practical 7.2: PointNet++ on ModelNet40 . . . 7

7 Practical 7.3: Voxel Grids and a 3D CNN . . . 8

7.1 A fault in the tutorial's voxelization . . . 8

7.2 Training on ModelNet10 . . . 8

8 Practical 7.4: MeshCNN on SHREC'16 . . . 9

9 Practical 7.5: Converting Between Representations . . . 10

10 Comparison and Discussion . . . 11

11 Limitations . . . 12

12 Conclusion . . . 12

References . . . 12

Appendix A Program Output . . . 13

A.1 Practical 7.1 and the 7.3 demo . . . 13

A.2 PointNet++ training (per-epoch summary) . . . 13

A.3 VoxelCNN training . . . 14

A.4 MeshCNN training (test accuracy per epoch) . . . 14

A.5 Practical 7.5: conversions . . . 15
Appendix B Source Code . . . 16
B.1 section_7_1/make_sphere.py (tutorial, unchanged) . . . 16
B.2 section_7_3/voxel_cnn_demo.py (tutorial, with the voxelization fix) . . . 16
B.3 section_7_3/voxelize_modelnet10.py . . . 17
B.4 section_7_3/train_voxel_cnn.py . . . 18
B.5 section_7_5/convert_representations.py . . . 19
B.6 prep_modelnet40.py . . . 21
B.7 make_figures.py . . . 21
B.8 Patch to Pointnet_Pointnet2_pytorch (CUDA to device-agnostic) . . . 24
B.9 Patch to MeshCNN (NumPy 1.24+ compatibility) . . . 25

1 Introduction

Images are regular 2D grids, so standard convolution applies to them directly. 3D shapes have no single natural format. A depth sensor or LiDAR produces an unordered set of points, a CAD model or a game asset is a triangle mesh, and medical scanners produce dense volumes. Each format makes some operations easy and others awkward, and the architectures used to learn from them differ for the same reason.

1.1 Objective

The objective is to run the practical part of the tutorial (its Section 7) end to end and to document what each step produces. The tutorial has five exercises, and each one is a section of this report:

- **7.1** Generate a synthetic point cloud and check that it can be written and read back (Section 5).
- **7.2** Train PointNet++ on ModelNet40 with the reference PyTorch implementation (Section 6).
- **7.3** Convert meshes to voxel grids and classify them with a 3D CNN (Section 7).
- **7.4** Train MeshCNN on the SHREC'16 mesh classification benchmark (Section 8).
- **7.5** Convert between meshes, point clouds and voxel grids (Section 9).

1.2 Contributions

Running the scripts exactly as printed does not work on current software and hardware, so the work goes slightly beyond transcription:

- The voxelization helper in the tutorial's `voxel_cnn_demo.py` returns non-cubic grids, which makes the 3D CNN crash. The fault is identified and fixed (Section 7.1).
- The 7.3 exercise stops at a single untrained forward pass. Here the same network is trained and evaluated on the full ModelNet10 dataset, so the voxel representation gets a real accuracy figure rather than a random prediction.
- Both third-party repositories are ported to current PyTorch and NumPy, and PointNet++ to the Apple MPS backend. The patches are minimal and listed in full in Appendix B.
- The 7.5 conversions are measured, not just run: each converted shape is scored against the original surface with the Chamfer distance.

2 Background

2.1 Point clouds: PointNet and PointNet++

A point cloud is a set $\{x_1, \ldots, x_n\}$ of 3D coordinates with no order and no connectivity. A network that consumes it must give the same answer for any permutation of the points. PointNet [1] achieves this by applying the same multi-layer perceptron h to every point independently and combining the results with a symmetric function, max-pooling:

$$f(\{x_1, \ldots, x_n\}) \approx \gamma\left(\max_i h(x_i)\right) \tag{1}$$

Because the max is taken over the whole set, PointNet sees each point either alone or as part of the global shape, and nothing in between, so it cannot learn local geometry. PointNet++ [2] applies PointNet hierarchically. Each *set abstraction* layer picks well-spread centre points with farthest point sampling, gathers the neighbours within a fixed radius of each centre (ball query), and summarises each neighbourhood with a small PointNet. The single-scale-grouping (SSG) classifier used here has three such layers: 512 centres with radius 0.2 and 32 neighbours, then 128 centres with radius 0.4 and 64 neighbours, then one global layer, followed by fully connected layers.

2.2 Voxel grids: 3D convolution

Rasterizing a shape into an $N \times N \times N$ occupancy grid makes it regular again, so ordinary convolution extends to 3D unchanged. VoxNet [3] and 3D ShapeNets [4] took this route. The price is memory and compute that grow with N^3, even though a surface occupies only a few percent of the cells (3.6% for the chair in Section 9). In practice this limits dense grids to about 32^3, which blurs thin parts. The tutorial's network is deliberately minimal: two 3×3×3 convolutions (32 and 64 channels), each followed by 2× max-pooling, then two fully connected layers. The first fully connected layer alone has 32,768×256 weights, so most of the model's parameters sit in it rather than in the convolutions.

2.3 Meshes: MeshCNN

A triangle mesh stores the surface exactly, together with its connectivity. MeshCNN [5] treats the *edges* as the basic elements. In a manifold mesh every edge borders exactly two triangles, so it has exactly four neighbouring edges, and this fixed neighbourhood lets convolution be defined much as it is on an image. Each edge starts with five features that do not change under rotation, translation or uniform scaling: the dihedral angle, the two inner angles opposite the edge, and two edge-length ratios. The four neighbours come in two pairs whose order is ambiguous, so the convolution combines each pair with symmetric functions (sum and absolute difference). Pooling is done by *edge collapse*: the network learns which edges to remove and simplifies the mesh while keeping the features that matter for the task.

Table 1. Trade-offs between the three representations (after Table 1 of the tutorial).

Representation	Strengths	Weaknesses	Typical uses
Point cloud	Native sensor output; compact; permutation-invariant networks are mature	No connectivity or surface; density varies	LiDAR and RGB-D perception, classification, segmentation
Voxel grid	Regular, so standard CNN machinery applies directly	Memory grows cubically; resolution is limited; most cells are empty	Medical volumes, occupancy prediction, low-resolution generation
Mesh	Exact surface with explicit connectivity; graphics-native	Irregular structure, so convolution must be redefined	Deformation, animation, topology-sensitive tasks

3 Datasets

Each exercise uses the dataset the tutorial names for it. Table 2 lists them.

Table 2. Datasets used. Splits are the standard ones distributed with each dataset.

Dataset	Used in	Format	Classes	Train / test	Notes
ModelNet40 [4] (normal-resampled)	7.2	10,000 points per shape (xyz + normals), text	40	9,843 / 2,468	First 1,024 points used, xyz only
ModelNet10 [4]	7.3, 7.5	Triangle meshes (.off)	10	3,991 / 908	Imbalanced: 106 to 889 training meshes per class
SHREC'16 [6] (MeshCNN split 16)	7.4	Triangle meshes (.obj), 750 edges each	30	480 / 120	Non-rigid shapes; 4 test meshes per class
Synthetic sphere	7.1	3,000 points, .xyz / .ply	-	-	Generated by the 7.1 script

The SHREC'16 test set is very small. One test mesh is worth 0.83 percentage points, so the per-epoch test accuracy in Section 8 is expected to fluctuate by several points between epochs even when training is stable.

4 Experimental Setup

4.1 Hardware and software

All experiments ran on a single Apple M4 laptop (10 CPU cores, integrated GPU) with no CUDA device. PointNet++ and the voxel CNN ran on the GPU through PyTorch's Metal (MPS) backend. MeshCNN ran on the CPU, because its code is written for CUDA or CPU only. Software: Python 3.11.15, PyTorch 2.15, trimesh 5.2, Open3D 0.20, NumPy 2.4 and SciPy 1.17. The tutorial targets Python 3.7 with PyTorch 1.6 and CUDA 10.1, so two small ports were needed:

- **PointNet++** (yanx27/Pointnet_Pointnet2_pytorch): every hard-coded `.cuda()` call is replaced with a device chosen at runtime (CUDA, then MPS, then CPU), and checkpoints are loaded with `map_location`. `test_classification.py` is also pointed at the same pre-sampled 1,024-point cache that training reads, because it otherwise insists on the raw .txt files, which were not kept. The model and training loop are unchanged.
- **MeshCNN** (ranahanocka/MeshCNN): NumPy 1.24 and later refuse to build arrays from lists of different lengths, which broke MeshCNN's mesh preprocessing cache. The per-vertex edge lists are now stored explicitly as an object array. The model and training loop are unchanged.

The normal-resampled ModelNet40 archive is about 7 GB once extracted. Rather than extract it, a short script (`prep_modelnet40.py`) streams each shape out of the zip file and writes the same 1,024-point cache file that the repository's loader would write with `--process_data`. The training data is therefore exactly what the unmodified loader would produce.

4.2 Training configuration

Table 3. Training configuration of each model.

	PointNet++ (SSG)	VoxelCNN	MeshCNN
Input	1,024 points, xyz	32^3 binary occupancy	750-edge mesh, 5 features per edge
Parameters	1.48 M	8.45 M	1.32 M
Optimizer	Adam, lr 1e-3, weight decay 1e-4	Adam, lr 1e-3, weight decay 1e-4	Adam, lr 2e-4
LR schedule	StepLR(×0.7 every 20 epochs)	StepLR(×0.5 every 5 epochs)	constant (the 100-epoch linear decay that follows was not run)
Batch size	24	32	16
Augmentation	random point dropout, scale, shift	random x/y mirroring, ±2-voxel shift	20 pre-computed augmentations per mesh: edge flips, vertex slides
Epochs	15 (tutorial: 200)	15 (tutorial: none, forward pass only)	100 of 200 (stopped at the end of the constant-LR phase)
Device	MPS (GPU)	MPS (GPU)	CPU

PointNet++ and MeshCNN use the repositories' default settings, apart from the epoch count. MeshCNN runs exactly the official `scripts/shrec/train.sh`.

The tutorial's 200 PointNet++ epochs were not feasible on this machine. One epoch, including its test pass, took about 13 minutes on average while the other jobs shared the machine, and still 7.5 minutes once it ran alone (epoch 15), so 200 epochs would take more than a day. Training was stopped after 15 epochs. The learning-rate schedule first decays at epoch 20, so this run is identical to running the script with `--epoch` 15. Most of the accuracy is reached early, as Figure 2 shows. The gap to the published figures is discussed in Section 6.

MeshCNN's default schedule is 100 epochs at a constant learning rate followed by 100 epochs of linear decay (about 4.5 hours on this CPU). It was stopped after epoch 100, at the end of the constant phase, so the run is a clean 100-epoch constant-rate schedule. Because of an off-by-one in the repository's schedule, epoch 100 itself ran at 0.99× the base rate, which is negligible. The decay phase was skipped; Section 8 shows that the training loss had already reached almost zero by then.

5 Practical 7.1: Synthetic Point Cloud

The script samples 3,000 directions uniformly on the sphere. It draws the azimuth φ uniformly in [0, 2π) and cos θ uniformly in [-1, 1], which gives equal density per unit area; drawing θ itself uniformly would crowd points at the poles. It then adds Gaussian noise with σ = 0.01 to imitate sensor error and writes the result as both .xyz and ASCII .ply. Reading the .ply back with Open3D returns all 3,000 points. Their distance from the origin has mean 0.9998 and standard deviation 0.0099, which matches the noise that was added (Figure 1). The write-read path works.

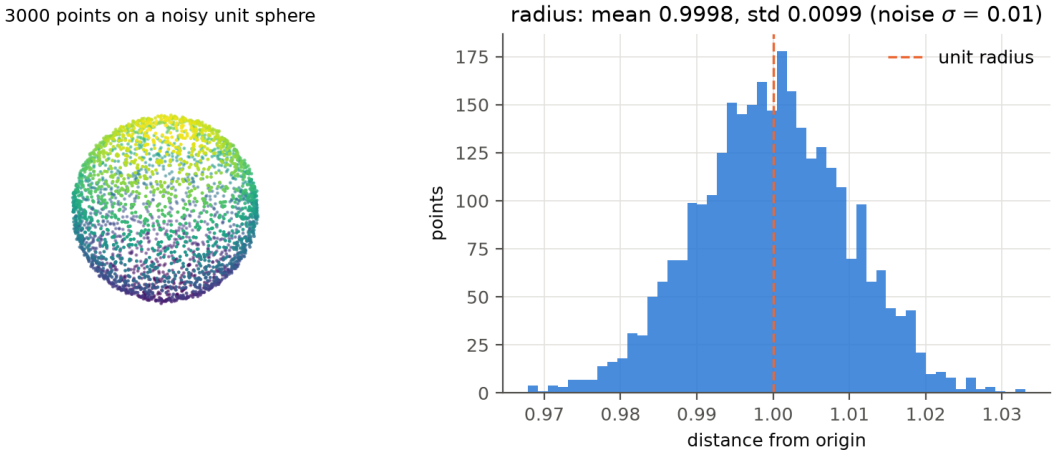


Figure 1. The synthetic point cloud from Practical 7.1 (left), and the distribution of point distances from the origin after reading the .ply back with Open3D (right).

6 Practical 7.2: PointNet++ on ModelNet40

Figure 2 shows the training curves. After a single epoch the model already classifies 68.3% of the test shapes correctly. Accuracy then rises more slowly and reaches a best of 88.2% instance accuracy at epoch 11, with a best class average of 82.8%. The class average is consistently lower than the instance accuracy because ModelNet40 is imbalanced. The test set has between 20 and 100 shapes per class, and the small classes, which the model learns least well, weigh as much as the large ones in the class average. The curve is not yet monotone: the last epoch scores 85.0%, so the best epoch again benefits from being picked on the test split (Section 11).

Re-evaluating the saved best checkpoint with the repository's `test_classification.py`, which averages three forward passes per shape (its default `--num_votes 3`; farthest point sampling starts from a random point, so the passes differ), gives 88.1% instance and 82.7% class accuracy. This is within one or two test shapes of the single-pass figure, so voting adds nothing measurable at this stage of training.

For comparison, the PointNet++ paper reports 90.7% with xyz input [2], and the repository reports 92.2% for this exact configuration after 200 epochs. The remaining gap comes from the shorter schedule: the learning rate never decays, and the augmentation has had far fewer passes over the data to take effect.

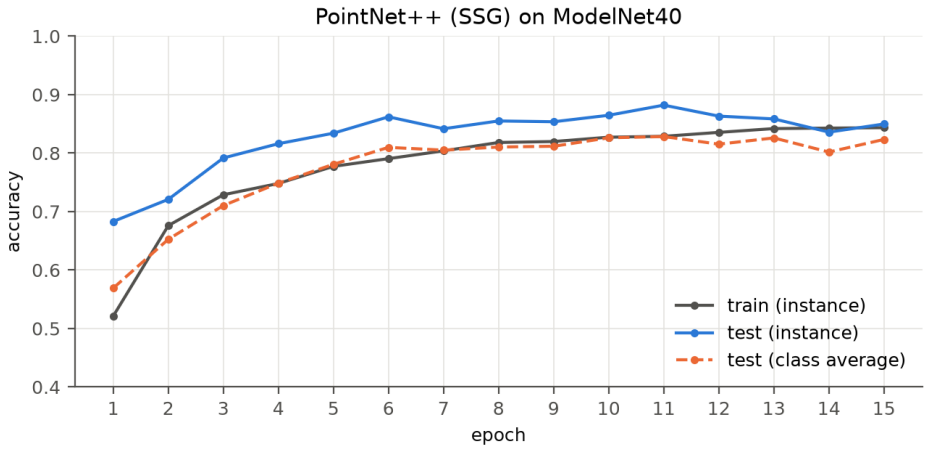


Figure 2. PointNet++ (SSG) training on ModelNet40: training accuracy, test accuracy and class-averaged test accuracy per epoch.

7 Practical 7.3: Voxel Grids and a 3D CNN

7.1 A fault in the tutorial's voxelization

Run as printed, `voxel_cnn_demo.py` fails inside the first fully connected layer:

```
RuntimeError: addmm: mat1 and mat2 shapes cannot be multiplied (1x59904 and 32768x256)
```

The cause is in `mesh_to_voxel`. `mesh.voxelized(pitch)` returns a grid that covers the shape's bounding box, which for the test chair is 15×17×25 voxels, not a cube. The code then rescales all three axes by the same factor, 32 / `shape[0]`, computed from the first axis alone. The result has the wrong size on the other two axes (59,904 / 64 channels = 936 cells after pooling, instead of $8^3 = 512$). The fix pads the grid with empty voxels so that it is centred in a cube, then rescales. This also keeps the shape's proportions, which per-axis rescaling would distort. An assertion now checks that the output is 32^3. After the fix the script produces a 1×1×32×32×32 input and 10 class scores, as the tutorial intends. The predicted class is arbitrary, because the network is untrained.

7.2 Training on ModelNet10

To get a meaningful result for the voxel representation, all 4,899 ModelNet10 meshes were voxelized at 32^3 with the corrected helper and cached to disk. Every mesh converted successfully. With 4 worker processes this took 12 minutes. The tutorial's network, unchanged, was then trained for 15 epochs. ModelNet10 is pose-aligned with z pointing up, so the only augmentation is random mirroring in the two horizontal directions and a small random shift.

Training took 15 minutes. The best test accuracy is **90.1%** at epoch 8, with a class average of 89.8% (Figure 3). Training accuracy keeps rising to 97.5% while test accuracy levels off around 88.3% after about five epochs. This is mild overfitting, as expected from 8.45 M parameters trained on 4,000 shapes. The confusion matrix (Figure 4) shows where the errors are. The weakest classes are *night_stand* (72%), *desk* (79%), *table* (80%). Of the 90 misclassified test shapes, 30 are confusions between *desk* and *table* and 23 between *night_stand* and *dresser*. These are boxy furniture classes that differ in proportions and small details. At 32^3, those details often take up only one or two voxels. For reference, 3D ShapeNets reported 83.5% and VoxNet 92.0% on ModelNet10 [3, 4], so this minimal two-layer network lands in the same range as the published voxel baselines.

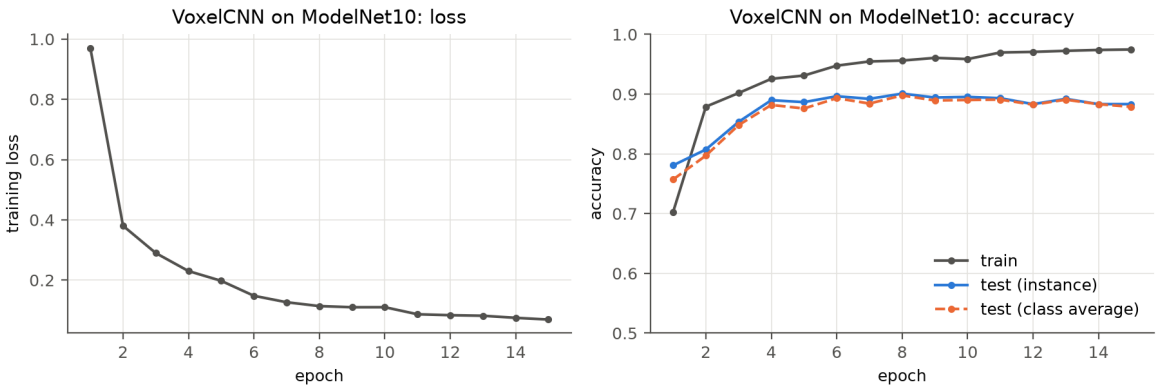


Figure 3. VoxelCNN training on ModelNet10 (32^3 occupancy): training loss (left) and accuracy (right).

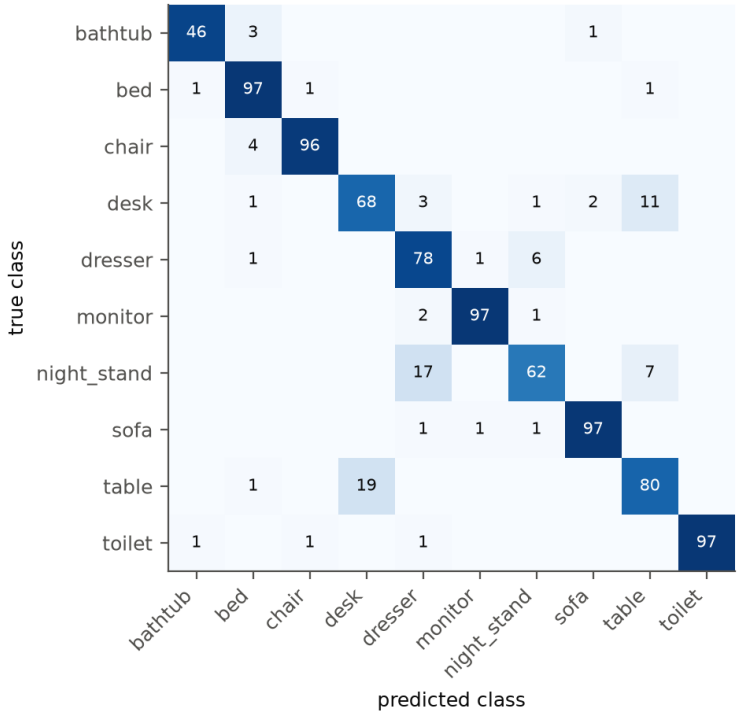


Figure 4. VoxelCNN confusion matrix on the 908 ModelNet10 test shapes at the best epoch (8). Cells give counts; colour is normalized per row.

8 Practical 7.4: MeshCNN on SHREC'16

MeshCNN was trained with the repository's own SHREC script. Each training mesh is precomputed in 20 augmented versions (random edge flips and vertex slides), and MeshCNN's pooling layers simplify each mesh from 750 edges down to 600, 450, 300 and finally 180. Figure 5 shows 100 epochs. Training loss falls steadily, and test accuracy rises from chance level (3.3% for 30 classes) to a best of **85.0%** at epoch 42. Between epochs 30 and 50 the accuracy still swings by more than ten points from one epoch to the next, so that peak is a single favourable epoch rather than a stable level. By epoch 60 the mean training loss has fallen below 0.01: the network fits every augmented training mesh, the gradients become tiny, and the test accuracy freezes. Over the last ten epochs it is 82.4% ± 0.2 points, that is, 99 of 120 test meshes in nine of the ten epochs. This plateau, not the earlier peak, is the honest summary of the run. The MeshCNN paper reports 98.6% on this split after the full 200 epochs [5]. The second 100 epochs only decay the learning rate, and with the training loss already near zero they are unlikely to close a 16-point gap on their own. A single run cannot isolate the cause; the seed, the CPU instead of a CUDA GPU, and newer library versions (including the NumPy patch of Section 4.1) are all candidates.

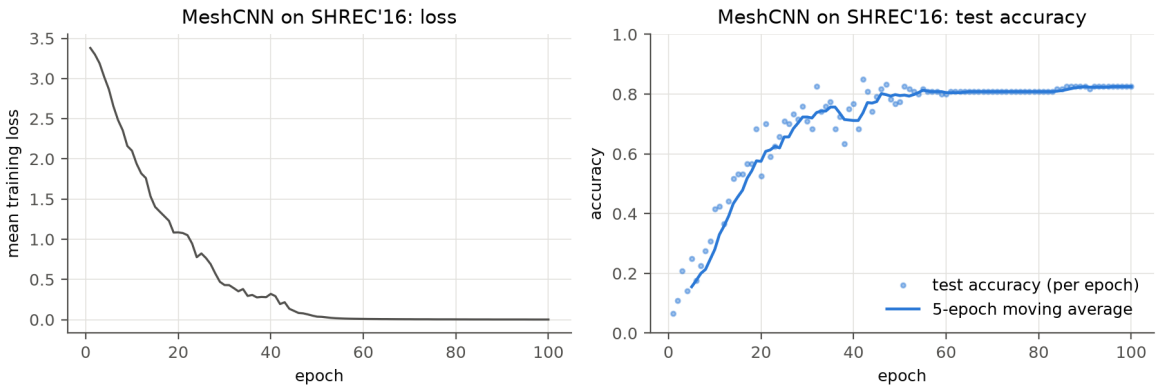


Figure 5. MeshCNN on SHREC'16 (split 16): mean training loss per epoch (left) and test accuracy (right). Each test point is one of 120 meshes, so accuracy moves in steps of 0.83 points; the line is a 5-epoch moving average.

9 Practical 7.5: Converting Between Representations

The ModelNet10 chair `chair_0001.off` (844 vertices, 2234 faces, not watertight) was converted along every path in the tutorial's conversion table (Figure 6):

- **Mesh to points:** 4,096 points sampled uniformly by area with `trimesh.sample.sample_surface`, as in the tutorial's snippet.
- **Mesh to voxels:** the corrected `mesh_to_voxel` at 32^3. 1191 of the 32,768 cells are occupied (3.6%).
- **Voxels to points:** the centres of the occupied voxels, mapped back into the mesh's coordinates.
- **Points to mesh:** Poisson surface reconstruction at octree depth 8, with the 5% of vertices least supported by the input points trimmed (39530 triangles), and Ball Pivoting with radii of 1.5, 3 and 6 times the mean point spacing (3136 triangles). Both methods need oriented normals, which were taken from the source faces.

Each result was scored against the original surface with the symmetric Chamfer distance, using 20,000 points sampled from each surface:

$$d_{\mathrm{CD}}(A, B) = \tfrac{1}{2} \left[d(A, B) + d(B, A) \right] \tag{2}$$

$$d(A, B) = \mathrm{mean}_{a \in A} \min_{b \in B} \| a - b \| \tag{3}$$

Table 4. Fidelity of each conversion to the original chair surface.

Conversion	Elements	Chamfer distance	Relative to object size
Voxel centres (32^3)	1191 points	0.638	1.74%
Poisson reconstruction	39530 triangles	0.881	2.40%
Ball Pivoting	3136 triangles	0.201	0.55%

Object size is the largest side of the bounding box (ModelNet units). Sampling is seeded, but Open3D's multithreaded Poisson and Ball Pivoting implementations vary slightly between runs (about 0.05 points of relative error); the ranking is unaffected.

The ranking follows from what each method assumes. Ball Pivoting only connects points that are already close together, so it never strays far from the true surface and has the lowest error. Where the thin tubular legs are sampled too sparsely, the ball falls through and leaves holes. Poisson reconstruction fits a closed, smooth surface. This chair is a thin folding frame and its mesh is not watertight, which violates that assumption: Poisson wraps the open tubes in a blob-like envelope and has the largest error despite using twelve times as many triangles. The voxel grid falls in between. At 32^3 each voxel is about 3% of the chair's size, so every thin bar becomes one or two voxels thick. This is the same loss of detail that makes desks and tables hard to tell apart in Section 7. The choice of conversion is therefore part of the modelling decision: a method that needs a watertight surface needs watertight input.

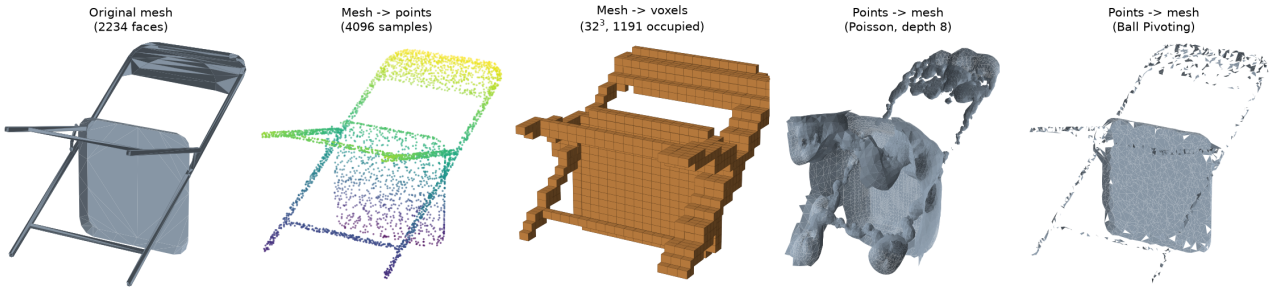


Figure 6. One ModelNet10 chair in every representation: the original mesh, 4,096 area-weighted surface samples, a 32^3 voxel grid, and meshes reconstructed from the samples with Poisson and Ball Pivoting.

10 Comparison and Discussion

Table 5. Summary of the three classifiers. Accuracies are on different datasets and are not directly comparable (see text).

	Point cloud	Voxel grid	Mesh
Model	PointNet++ (SSG)	VoxelCNN (tutorial)	MeshCNN
Dataset	ModelNet40 (40 classes)	ModelNet10 (10 classes)	SHREC'16 (30 classes)
Parameters	1.48 M	8.45 M	1.32 M
Epochs run	15	15	100
Time per epoch	13.2 min (GPU)	1.0 min (GPU)	1.4 min (CPU)
Time per shape	64 ms	12 ms	143 ms
Test accuracy	88.1% instance, 82.7% class	90.1% instance, 89.8% class	85.0% best, 82.4% last-10 mean
Published reference	90.7% [2], 92.2% (repository)	83.5% [4], 92.0% [3]	98.6% [5]

Times were measured while the three jobs shared the same machine, so they show the relative order of cost rather than each model's best-case speed.

The three accuracies cannot be ranked against each other. The datasets differ in the number of classes (10, 30 and 40), in the kind of shapes (rigid furniture versus non-rigid animals and people) and in size (480 to 9,843 training shapes). What the experiments do show is the cost and behaviour of each representation:

- **Parameter efficiency.** PointNet++ and MeshCNN each use about 1.4 M parameters. The two-layer voxel network uses six times as many, and 99.3% of them are in its first fully connected layer, which has to read a dense 64×8^3 feature volume of which most is empty space. This is the cubic cost of Section 2.2 appearing in practice.
- **Compute per shape.** The voxel CNN is by far the cheapest (12 ms per shape), because dense 3D convolution maps directly onto GPU hardware. PointNet++ costs about 5 times more (64 ms) despite having six times fewer parameters. Its farthest point sampling is a sequential loop of 512 steps per batch in this implementation, and each step is a tiny GPU operation whose launch overhead dominates. MeshCNN is the most expensive per shape (143 ms), but it ran on the CPU, so this figure is not directly comparable. Its cost is dominated by irregular mesh bookkeeping (edge collapse during pooling, loading the augmented meshes), which is hard to parallelize on any device.
- **What each one discards.** At 32^3 the voxel grid loses thin structures (Section 9), and the voxel CNN's main confusions are exactly between classes that differ in such details. The point cloud keeps fine geometry but loses connectivity, so recovering a surface takes an extra step with its own assumptions. The mesh keeps both, which is why MeshCNN can tell apart non-rigid SHREC shapes from their intrinsic geometry. It requires clean manifold input, however: the repository's SHREC data has been remeshed to exactly 750 edges per shape.

These observations agree with the tutorial's practical advice. Point-based networks such as PointNet++ are the default starting point. Voxels make sense when the data is inherently volumetric or the resolution needed is low. Mesh networks pay off when the surface and its topology carry the signal.

11 Limitations

- Because of the hardware, PointNet++ was trained for 15 of the tutorial's 200 epochs, so its accuracy is a lower bound on what this configuration reaches. MeshCNN ran 100 of its default 200 epochs, without the learning-rate decay; it had plateaued, but it remains well below the published figure for reasons this single run cannot separate.
- Each model was trained once with one seed. With 120 test meshes, the uncertainty in the SHREC'16 accuracy is several points; repeated runs would give proper error bars.
- All three models are evaluated on the training run's own test split, and the best epoch is chosen on that same split, with no separate validation set. This is the protocol both repositories use, but it slightly favours the reported numbers.
- The conversions in Section 9 were measured on one shape. It is a deliberately hard case (thin and not watertight), and a closed, chunky shape would favour Poisson reconstruction.

12 Conclusion

All five practical exercises of the tutorial were completed on a laptop without a CUDA GPU, after small porting fixes to both repositories and a fix to the tutorial's own voxelization code. PointNet++ reached 88.1% on ModelNet40 in 15 epochs. The tutorial's voxel CNN, extended from a forward pass to full training, reached 90.1% on ModelNet10. MeshCNN peaked at 85.0% on SHREC'16 and settled at 82.4%. The conversion experiment showed directly the detail each representation discards. Ball Pivoting stayed within 0.6% of the object's size, while Poisson reconstruction, whose closed-surface assumption fails on a thin open frame, was off by 2.4%. Choosing a 3D representation determines which network can be used, what it costs to train, and which geometric details are available to it at all.

References

[1] C. R. Qi, H. Su, K. Mo and L. J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *Proc. IEEE CVPR*, 2017.

[2] C. R. Qi, L. Yi, H. Su and L. J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] D. Maturana and S. Scherer. VoxNet: A 3D Convolutional Neural Network for Real-Time Object Recognition. In *Proc. IEEE/RSJ IROS*, 2015.

[4] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang and J. Xiao. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proc. IEEE CVPR*, 2015.

[5] R. Hanocka, A. Hertz, N. Fish, R. Giryes, S. Fleishman and D. Cohen-Or. MeshCNN: A Network with an Edge. *ACM Transactions on Graphics (SIGGRAPH)*, 38(4), 2019.

[6] Z. Lian et al. SHREC'11 Track: Shape Retrieval on Non-rigid 3D Watertight Meshes. In *Eurographics Workshop on 3D Object Retrieval*, 2011. (Dataset distributed with MeshCNN as SHREC'16.)

[7] M. Kazhdan, M. Bolitho and H. Hoppe. Poisson Surface Reconstruction. In *Proc. Eurographics Symposium on Geometry Processing*, 2006.

[8] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva and G. Taubin. The Ball-Pivoting Algorithm for Surface Reconstruction. *IEEE Transactions on Visualization and Computer Graphics*, 5(4):349-359, 1999.

[9] X. Yan. Pointnet_Pointnet2_pytorch. https://github.com/yanx27/Pointnet_Pointnet2_pytorch, 2019.

Appendix A Program Output

A.1 Practical 7.1 and the 7.3 demo

```
$ python section_7_1/make_sphere.py
Created data/sphere.ply and data/sphere.xyz
  3000 points on a noisy unit sphere
loaded via open3d: (3000, 3) radius mean 0.9998 std 0.0099

$ python section_7_3/voxel_cnn_demo.py      # after the fix
Input shape: torch.Size([1, 1, 32, 32, 32])
Output logits shape: torch.Size([1, 10])
Predicted class index: 5
raw voxelized shape: (15, 17, 25)
fixed grid: (32, 32, 32) occupied: 1191
```

A.2 PointNet++ training (per-epoch summary)

```
$ python train_classification.py --model pointnet2_cls_ssg --log_dir pointnet2_modelnet40 \
      --num_category 40 --epoch 25 --batch_size 24 --process_data   # stopped after epoch 15
2026-10-30 13:18:29,147  Train Instance Accuracy: 0.748069
2026-10-30 13:20:50,545  Test Instance Accuracy: 0.816100, Class Accuracy: 0.748669
2026-10-30 13:20:50,546  Best Instance Accuracy: 0.816100, Class Accuracy: 0.748669
2026-10-30 13:20:50,726  Epoch 5 (5/25):
2026-10-30 13:28:30,020  Train Instance Accuracy: 0.777337
2026-10-30 13:30:24,290  Test Instance Accuracy: 0.834061, Class Accuracy: 0.780832
2026-10-30 13:30:24,291  Best Instance Accuracy: 0.834061, Class Accuracy: 0.780832
2026-10-30 13:30:24,534  Epoch 6 (6/25):
2026-10-30 13:44:24,376  Train Instance Accuracy: 0.790549
2026-10-30 14:14:23,952  Test Instance Accuracy: 0.862055, Class Accuracy: 0.809843
2026-10-30 14:14:23,952  Best Instance Accuracy: 0.862055, Class Accuracy: 0.809843
2026-10-30 14:14:24,199  Epoch 7 (7/25):
2026-10-30 14:20:52,505  Train Instance Accuracy: 0.803862
2026-10-30 14:22:41,541  Test Instance Accuracy: 0.841667, Class Accuracy: 0.805075
2026-10-30 14:22:41,543  Best Instance Accuracy: 0.862055, Class Accuracy: 0.809843
2026-10-30 14:22:41,543  Epoch 8 (8/25):
2026-10-30 14:32:20,377  Train Instance Accuracy: 0.818191
2026-10-30 14:34:09,937  Test Instance Accuracy: 0.855016, Class Accuracy: 0.810434
2026-10-30 14:34:09,938  Best Instance Accuracy: 0.862055, Class Accuracy: 0.810434
2026-10-30 14:34:09,938  Epoch 9 (9/25):
2026-10-30 14:46:08,864  Train Instance Accuracy: 0.819817
2026-10-30 14:47:50,475  Test Instance Accuracy: 0.853641, Class Accuracy: 0.811749
2026-10-30 14:47:50,476  Best Instance Accuracy: 0.862055, Class Accuracy: 0.811749
2026-10-30 14:47:50,477  Epoch 10 (10/25):
2026-10-30 14:53:32,982  Train Instance Accuracy: 0.827236
2026-10-30 14:55:13,636  Test Instance Accuracy: 0.864725, Class Accuracy: 0.826187
2026-10-30 14:55:13,637  Best Instance Accuracy: 0.864725, Class Accuracy: 0.826187
2026-10-30 14:55:13,865  Epoch 11 (11/25):
2026-10-30 15:00:55,884  Train Instance Accuracy: 0.828760
2026-10-30 15:02:35,542  Test Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:02:35,544  Best Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:02:35,724  Epoch 12 (12/25):
2026-10-30 15:08:19,014  Train Instance Accuracy: 0.835467
2026-10-30 15:10:00,453  Test Instance Accuracy: 0.863269, Class Accuracy: 0.815285
2026-10-30 15:10:00,454  Best Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:10:00,455  Epoch 13 (13/25):
2026-10-30 15:15:43,379  Train Instance Accuracy: 0.841972
2026-10-30 15:17:24,015  Test Instance Accuracy: 0.858495, Class Accuracy: 0.825969
2026-10-30 15:17:24,016  Best Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:17:24,016  Epoch 14 (14/25):
2026-10-30 15:28:13,428  Train Instance Accuracy: 0.842785
2026-10-30 15:31:15,481  Test Instance Accuracy: 0.835922, Class Accuracy: 0.801841
2026-10-30 15:31:15,482  Best Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:31:15,482  Epoch 15 (15/25):
2026-10-30 15:37:04,003  Train Instance Accuracy: 0.843496
2026-10-30 15:38:45,475  Test Instance Accuracy: 0.849757, Class Accuracy: 0.823392
2026-10-30 15:38:45,477  Best Instance Accuracy: 0.882039, Class Accuracy: 0.828320
2026-10-30 15:38:45,477  Epoch 16 (16/25):
```

A.3 VoxelCNN training

```
$ python section_7_3/voxelize_modelnet10.py 4
train: 3991 / 3991 meshes voxelized
test: 908 / 908 meshes voxelized
wrote /Users/aggimallaabhishek/Downloads/College/3D Computer Vision/Assignment 3/section_7_3/data/modelnet10_vox32....

$ python section_7_3/train_voxel_cnn.py 15
train (3991, 32, 32, 32), test (908, 32, 32, 32), device mps
parameters: 8.45M
epoch  1  loss 0.971  train 0.703  test 0.781  class-avg 0.757  (65s)
epoch  2  loss 0.380  train 0.879  test 0.807  class-avg 0.797  (123s)
epoch  3  loss 0.290  train 0.902  test 0.854  class-avg 0.848  (179s)
epoch  4  loss 0.230  train 0.926  test 0.890  class-avg 0.882  (234s)
epoch  5  loss 0.198  train 0.931  test 0.887  class-avg 0.876  (291s)
epoch  6  loss 0.148  train 0.948  test 0.896  class-avg 0.893  (345s)
epoch  7  loss 0.127  train 0.955  test 0.892  class-avg 0.884  (404s)
epoch  8  loss 0.114  train 0.956  test 0.901  class-avg 0.898  (469s)
epoch  9  loss 0.110  train 0.961  test 0.894  class-avg 0.889  (532s)
epoch 10  loss 0.110  train 0.959  test 0.895  class-avg 0.890  (590s)
epoch 11  loss 0.087  train 0.970  test 0.893  class-avg 0.891  (649s)
epoch 12  loss 0.084  train 0.971  test 0.883  class-avg 0.882  (708s)
epoch 13  loss 0.082  train 0.972  test 0.892  class-avg 0.891  (766s)
epoch 14  loss 0.075  train 0.974  test 0.883  class-avg 0.883  (822s)
epoch 15  loss 0.069  train 0.975  test 0.883  class-avg 0.879  (876s)
best epoch 8: test 0.901, class-avg 0.898
  bathtub      0.920
  bed          0.970
  chair        0.960
  desk         0.791
  dresser      0.907
  monitor      0.970
  night_stand  0.721
  sofa         0.970
  table        0.800
  toilet       0.970
wrote figures and outputs/voxel_cnn_results.json
```

A.4 MeshCNN training (test accuracy per epoch)

```
$ python train.py --dataroot datasets/shrec_16 --name shrec16 --ncf 64 128 256 256 \
      --pool_res 600 450 300 180 --norm group --resblocks 1 --flip_edges 0.2 \
      --slide_verts 0.2 --num_aug 20 --niter_decay 100 --gpu_ids -1 --no_vis --seed 0
epoch: 61, TEST ACC: [80.833 %]
epoch: 62, TEST ACC: [80.833 %]
epoch: 63, TEST ACC: [80.833 %]
epoch: 64, TEST ACC: [80.833 %]
epoch: 65, TEST ACC: [80.833 %]
epoch: 66, TEST ACC: [80.833 %]
epoch: 67, TEST ACC: [80.833 %]
epoch: 68, TEST ACC: [80.833 %]
epoch: 69, TEST ACC: [80.833 %]
epoch: 70, TEST ACC: [80.833 %]
epoch: 71, TEST ACC: [80.833 %]
epoch: 72, TEST ACC: [80.833 %]
epoch: 73, TEST ACC: [80.833 %]
epoch: 74, TEST ACC: [80.833 %]
epoch: 75, TEST ACC: [80.833 %]
epoch: 76, TEST ACC: [80.833 %]
epoch: 77, TEST ACC: [80.833 %]
epoch: 78, TEST ACC: [80.833 %]
epoch: 79, TEST ACC: [80.833 %]
epoch: 80, TEST ACC: [80.833 %]
epoch: 81, TEST ACC: [80.833 %]
epoch: 82, TEST ACC: [80.833 %]
epoch: 83, TEST ACC: [80.833 %]
epoch: 84, TEST ACC: [81.667 %]
epoch: 85, TEST ACC: [81.667 %]
epoch: 86, TEST ACC: [82.5 %]
epoch: 87, TEST ACC: [82.5 %]
epoch: 88, TEST ACC: [82.5 %]
epoch: 89, TEST ACC: [82.5 %]
epoch: 90, TEST ACC: [82.5 %]
epoch: 91, TEST ACC: [81.667 %]
epoch: 92, TEST ACC: [82.5 %]
epoch: 93, TEST ACC: [82.5 %]
epoch: 94, TEST ACC: [82.5 %]
epoch: 95, TEST ACC: [82.5 %]
epoch: 96, TEST ACC: [82.5 %]
epoch: 97, TEST ACC: [82.5 %]
epoch: 98, TEST ACC: [82.5 %]
epoch: 99, TEST ACC: [82.5 %]
epoch: 100, TEST ACC: [82.5 %]
```

A.5 Practical 7.5: conversions

```
$ python section_7_5/convert_representations.py
mesh: 844 vertices, 2234 faces, watertight = False
mesh -> points: (4096, 3)
mesh -> voxels: (32, 32, 32) occupied = 1191 (3.6% of grid)
voxels -> points: (1191, 3)
points -> mesh (Poisson): 19868 vertices, 39530 triangles
points -> mesh (Ball Pivoting): 4096 vertices, 3136 triangles
  Poisson       Chamfer distance = 0.8814 (2.40% of object size)
  Ball Pivoting Chamfer distance = 0.2009 (0.55% of object size)
  voxel centres  Chamfer distance = 0.6382 (1.74% of object size)
wrote figures/representations.png
```

Appendix B Source Code

The scripts written for this assignment are listed below, followed by the patches applied to the two cloned repositories. Steps to reproduce:

```
python -m venv venv && venv/bin/pip install torch trimesh open3d scipy scikit-image matplotlib tqdm reportlab
python prep_modelnet40.py data/modelnet40_normal_resampled.zip       # ModelNet40 cache for 7.2
(cd section_7_1 && python make_sphere.py)                            # 7.1
(cd Pointnet_Pointnet2_pytorch && python train_classification.py --model pointnet2_cls_ssg \
    --log_dir pointnet2_modelnet40 --num_category 40 --epoch 15 --batch_size 24 --process_data \
    && python test_classification.py --log_dir pointnet2_modelnet40)  # 7.2
(cd section_7_3 && python voxel_cnn_demo.py && python voxelize_modelnet10.py && python train_voxel_cnn.py)
(cd MeshCNN && bash scripts/shrec/get_data.sh && bash scripts/shrec/train.sh)   # 7.4 (+ --gpu_ids -1 on CPU)
python section_7_5/convert_representations.py                        # 7.5
python make_figures.py && python build_report.py                     # figures and this PDF
```

B.1 section_7_1/make_sphere.py (tutorial, unchanged)

```python
import os
import numpy as np
# Generate 3000 points uniformly distributed on a unit sphere
n = 3000
phi = np.random.uniform(0, 2 * np.pi, n)
cos_theta = np.random.uniform(-1, 1, n)
sin_theta = np.sqrt(1 - cos_theta**2)
x = sin_theta * np.cos(phi)
y = sin_theta * np.sin(phi)
z = cos_theta
pts = np.stack([x, y, z], axis=1)
pts += np.random.normal(0, 0.01, pts.shape)  # Sensor noise
os.makedirs("data", exist_ok=True)
# 1. Save as XYZ
np.savetxt("data/sphere.xyz", pts, fmt="%.6f")
# 2. Save as clean ASCII PLY
ply_header = (
    "ply\n"
    "format ascii 1.0\n"
    f"element vertex {len(pts)}\n"
    "property float x\n"
    "property float y\n"
    "property float z\n"
    "end_header\n"
)
with open("data/sphere.ply", "w") as f:
    f.write(ply_header)
    np.savetxt(f, pts, fmt="%.6f")
print(f"Created data/sphere.ply and data/sphere.xyz")
print(f"  {len(pts)} points on a noisy unit sphere")
```

B.2 section_7_3/voxel_cnn_demo.py (tutorial, with the voxelization fix)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import trimesh
from scipy.ndimage import zoom
# ------------------------------------------------------------
# 1. Mesh-to-voxel conversion
# ------------------------------------------------------------
def mesh_to_voxel(file_path, grid_size=32):
    """
    Load a mesh and rasterize it into a dense occupancy grid.
    Returns a float32 array of shape (grid_size, grid_size, grid_size).
    """
    # Also accepts an already-loaded trimesh.Trimesh (voxelize_modelnet10.py patches bad .off headers first)
    mesh = file_path if isinstance(file_path, trimesh.Trimesh) else trimesh.load(file_path)
    # Voxelize at a pitch that yields roughly grid_size cells along the largest axis
    pitch = mesh.scale / grid_size
    voxel = mesh.voxelized(pitch=pitch).matrix.astype(np.float32)
    # voxelized() returns the object's bounding box (e.g. 8x9x13), not a cube, so scaling by
    # shape[0] alone leaves a non-cubic grid. Centre it in a cube first to keep the aspect ratio.
    side = max(voxel.shape)
    pad = [((side - s) // 2, side - s - (side - s) // 2) for s in voxel.shape]
    voxel = np.pad(voxel, pad)
    # Resize to exactly grid_size^3 (nearest-neighbor via zoom)
    scale_factor = grid_size / side
    voxel = zoom(voxel, (scale_factor,) * 3, order=0)  # order=0 = nearest
    assert voxel.shape == (grid_size,) * 3, voxel.shape
    return voxel
# ------------------------------------------------------------
# 2. Simple 3D CNN for classification
# ------------------------------------------------------------
class VoxelCNN(nn.Module):
```

```
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv3d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv3d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool3d(2)
        # After two poolings on a 32^3 input: 32 -> 16 -> 8
        self.fc1 = nn.Linear(64 * 8 * 8 * 8, 256)
        self.fc2 = nn.Linear(256, num_classes)
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))   # (B, 32, 16, 16, 16)
        x = self.pool(F.relu(self.conv2(x)))   # (B, 64, 8, 8, 8)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
# ------------------------------------------------------------
# 3. Demo: load a mesh, convert, run forward pass
# ------------------------------------------------------------
if __name__ == "__main__":
    # Replace with an actual .off or .obj file from ModelNet10
    mesh_path = "../data/ModelNet10/chair/train/chair_0001.off"
    vox = mesh_to_voxel(mesh_path, grid_size=32)   # (32, 32, 32)
    # Add batch and channel dims: (1, 1, 32, 32, 32)
    vox_tensor = torch.tensor(vox).unsqueeze(0).unsqueeze(0)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = VoxelCNN(num_classes=10).to(device)
    output = model(vox_tensor.to(device))
    print("Input shape:", vox_tensor.shape)
    print("Output logits shape:", output.shape)
    print("Predicted class index:", output.argmax(dim=1).item())
```

B.3 section_7_3/voxelize_modelnet10.py

```
"""Voxelizes every ModelNet10 mesh at 32^3 with the 7.3 mesh_to_voxel helper and caches the grids.

Output: section_7_3/data/modelnet10_vox32.npz with uint8 grids X_train/X_test, labels y_train/y_test and class names.
load_off guards against .off files that glue the counts onto the header line ("OFF490 518 0"), a known ModelNet
quirk; every ModelNet10 file turned out to be well-formed, so here it is only a safeguard.
"""
import glob
import io
import os
import sys
import time
from multiprocessing import Pool

import numpy as np
import trimesh
from tqdm import tqdm

from voxel_cnn_demo import mesh_to_voxel

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.join(HERE, "..", "data", "ModelNet10")
OUT = os.path.join(HERE, "data", "modelnet10_vox32.npz")
WORKERS = int(sys.argv[1]) if len(sys.argv) > 1 else 4


def load_off(path):
    with open(path) as f:
        text = f.read()
    if text.startswith("OFF") and not text.startswith("OFF\n"):
        text = "OFF\n" + text[3:].lstrip()
    return trimesh.load(io.StringIO(text), file_type="off")


def worker(path):
    try:
        return (mesh_to_voxel(load_off(path), grid_size=32) > 0).astype(np.uint8)
    except Exception as e:  # report and skip rather than kill the whole pool
        print("FAILED", path, e, flush=True)
        return None


if __name__ == "__main__":
    classes = sorted(d for d in os.listdir(ROOT) if os.path.isdir(os.path.join(ROOT, d)))
    os.makedirs(os.path.dirname(OUT), exist_ok=True)
    arrays = {"classes": np.array(classes)}
    t0 = time.time()
    for split in ["train", "test"]:
        files = [(f, i) for i, c in enumerate(classes) for f in sorted(glob.glob(os.path.join(ROOT, c, split, "*.off")))]
        with Pool(WORKERS) as pool:
            grids = list(tqdm(pool.imap(worker, [f for f, _ in files], chunksize=8), total=len(files), desc=split))
        keep = [k for k, g in enumerate(grids) if g is not None]
```

```
        arrays["X_" + split] = np.stack([grids[k] for k in keep])
        arrays["y_" + split] = np.array([files[k][1] for k in keep], dtype=np.int64)
        print("%s: %d / %d meshes voxelized" % (split, len(keep), len(files)))
    np.savez_compressed(OUT, **arrays)
    print("wrote %s in %.0fs" % (OUT, time.time() - t0))
```

B.4 section_7_3/train_voxel_cnn.py

```
"""Trains the tutorial's VoxelCNN (unchanged from voxel_cnn_demo.py) on the cached 32^3 ModelNet10 grids.

Augmentation: random mirroring along the two horizontal axes and a +-2 voxel shift. ModelNet10 is pose-aligned with
z up, so no rotation is applied. Reports overall and per-class test accuracy each epoch, keeps the best checkpoint,
and writes figures/voxel_cnn_curves.png, figures/voxel_cnn_confusion.png and outputs/voxel_cnn_results.json.

usage: python train_voxel_cnn.py [epochs]
"""
import json
import os
import sys
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F

from voxel_cnn_demo import VoxelCNN

HERE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(HERE, "data", "modelnet10_vox32.npz")
FIGS = os.path.join(HERE, "..", "figures")
OUTS = os.path.join(HERE, "..", "outputs")
EPOCHS = int(sys.argv[1]) if len(sys.argv) > 1 else 15
BATCH = 32
torch.manual_seed(0)
np.random.seed(0)
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")

d = np.load(DATA)
classes = list(d["classes"])
X_train, y_train = torch.from_numpy(d["X_train"]).float(), torch.from_numpy(d["y_train"])
X_test, y_test = torch.from_numpy(d["X_test"]).float(), torch.from_numpy(d["y_test"])
print("train %s, test %s, device %s" % (tuple(X_train.shape), tuple(X_test.shape), device))


def augment(x):
    # x: (B, 32, 32, 32) indexed (x, y, z); mirror x and y independently per sample, then shift
    flip_x = torch.rand(x.shape[0]) < 0.5
    flip_y = torch.rand(x.shape[0]) < 0.5
    x[flip_x] = x[flip_x].flip(1)
    x[flip_y] = x[flip_y].flip(2)
    sx, sy, sz = np.random.randint(-2, 3, size=3)
    return torch.roll(x, shifts=(int(sx), int(sy), int(sz)), dims=(1, 2, 3))


@torch.no_grad()
def evaluate(model):
    model.eval()
    preds = torch.cat([model(X_test[i:i + 128].unsqueeze(1).to(device)).argmax(1).cpu()
                       for i in range(0, len(X_test), 128)])
    conf = np.zeros((len(classes), len(classes)), dtype=int)
    np.add.at(conf, (y_test.numpy(), preds.numpy()), 1)
    return (preds == y_test).float().mean().item(), (conf.diagonal() / conf.sum(1)).mean(), conf


model = VoxelCNN(num_classes=len(classes)).to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=max(1, EPOCHS // 3), gamma=0.5)
print("parameters: %.2fM" % (sum(p.numel() for p in model.parameters()) / 1e6))

history, best = [], {"test_acc": -1}
t0 = time.time()
for epoch in range(1, EPOCHS + 1):
    model.train()
    perm = torch.randperm(len(X_train))
    loss_sum, correct = 0.0, 0
    for i in range(0, len(perm), BATCH):
        idx = perm[i:i + BATCH]
        x = augment(X_train[idx].clone()).unsqueeze(1).to(device)
        y = y_train[idx].to(device)
        out = model(x)
        loss = F.cross_entropy(out, y)
```

```
        opt.zero_grad()
        loss.backward()
        opt.step()
        loss_sum += loss.item() * len(idx)
        correct += (out.argmax(1) == y).sum().item()
    sched.step()
    test_acc, class_acc, conf = evaluate(model)
    history.append({"epoch": epoch, "loss": loss_sum / len(perm), "train_acc": correct / len(perm),
                    "test_acc": test_acc, "test_class_acc": float(class_acc)})
    print("epoch %2d  loss %.3f  train %.3f  test %.3f  class-avg %.3f  (%.0fs)" % (
        epoch, history[-1]["loss"], history[-1]["train_acc"], test_acc, class_acc, time.time() - t0), flush=
True)
    if test_acc > best["test_acc"]:
        best = {"epoch": epoch, "test_acc": test_acc, "test_class_acc": float(class_acc), "confusion": conf.
tolist()}
        torch.save(model.state_dict(), os.path.join(HERE, "data", "voxel_cnn_best.pth"))

os.makedirs(FIGS, exist_ok=True)
os.makedirs(OUTS, exist_ok=True)
conf = np.array(best["confusion"])
best["per_class_acc"] = {c: float(conf[k, k] / conf[k].sum()) for k, c in enumerate(classes)}
with open(os.path.join(OUTS, "voxel_cnn_results.json"), "w") as f:
    json.dump({"classes": classes, "epochs": EPOCHS, "batch_size": BATCH, "train_seconds": time.time() - t0,
               "best": best, "history": history}, f, indent=2)
print("best epoch %d: test %.3f, class-avg %.3f" % (best["epoch"], best["test_acc"], best["test_class_acc"])
)
for c, a in best["per_class_acc"].items():
    print("  %-12s %.3f" % (c, a))

ep = [h["epoch"] for h in history]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 3.6))
a1.plot(ep, [h["loss"] for h in history], "o-", ms=3)
a1.set(xlabel="epoch", ylabel="train loss", title="VoxelCNN on ModelNet10 (32$^3$)")
a2.plot(ep, [h["train_acc"] for h in history], "o-", ms=3, label="train")
a2.plot(ep, [h["test_acc"] for h in history], "o-", ms=3, label="test (overall)")
a2.plot(ep, [h["test_class_acc"] for h in history], "o--", ms=3, label="test (class avg)")
a2.set(xlabel="epoch", ylabel="accuracy", ylim=(0, 1))
a2.legend()
for a in (a1, a2):
    a.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIGS, "voxel_cnn_curves.png"), dpi=150)

fig, ax = plt.subplots(figsize=(6.4, 5.6))
norm = conf / conf.sum(1, keepdims=True)
ax.imshow(norm, cmap="Blues", vmin=0, vmax=1)
for r in range(len(classes)):
    for c in range(len(classes)):
        if conf[r, c]:
            ax.text(c, r, conf[r, c], ha="center", va="center", fontsize=8, color="white" if norm[r, c] > 0.
5 else "black")
ax.set_xticks(range(len(classes)), classes, rotation=45, ha="right")
ax.set_yticks(range(len(classes)), classes)
ax.set(xlabel="predicted", ylabel="true", title="VoxelCNN confusion (best epoch %d, acc %.1f%%)" % (best["ep
och"], 100 * best["test_acc"]))
plt.tight_layout()
plt.savefig(os.path.join(FIGS, "voxel_cnn_confusion.png"), dpi=150)
print("wrote figures and outputs/voxel_cnn_results.json")
```

B.5 section_7_5/convert_representations.py

```
"""Section 7.5: round-trip one ModelNet10 chair through every representation in the tutorial's table.

mesh -> point cloud (area-weighted surface sampling, trimesh)
mesh -> voxel grid  (trimesh.voxelized, via the 7.3 helper)
voxel -> point cloud (occupied voxel centres)
point cloud -> mesh (Poisson and Ball Pivoting, open3d)

Writes the intermediate files to section_7_5/data/ and a side-by-side figure to figures/representations.png.
"""
import os
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import open3d as o3d
import trimesh

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "section_7_3"))
from voxel_cnn_demo import mesh_to_voxel

HERE = os.path.dirname(os.path.abspath(__file__))
MESH = os.path.join(HERE, "..", "data", "ModelNet10", "chair", "train", "chair_0001.off")
OUT = os.path.join(HERE, "data")
```

```
FIG = os.path.join(HERE, "..", "figures", "representations.png")
os.makedirs(OUT, exist_ok=True)
os.makedirs(os.path.dirname(FIG), exist_ok=True)
np.random.seed(0)  # trimesh sampling takes its own seed= below

mesh = trimesh.load(MESH)
print("mesh:", len(mesh.vertices), "vertices,", len(mesh.faces), "faces, watertight =", mesh.is_watertight)

# 1. Mesh -> point cloud (the tutorial's minimal sampler)
points, face_indices = trimesh.sample.sample_surface(mesh, 4096, seed=0)
np.savetxt(os.path.join(OUT, "chair_sampled.xyz"), points, fmt="%.6f")
print("mesh -> points:", points.shape)

# 2. Mesh -> voxel grid
vox = mesh_to_voxel(MESH, grid_size=32)
print("mesh -> voxels:", vox.shape, "occupied =", int(vox.sum()), "(%.1f%% of grid)" % (100 * vox.mean()))

# 3. Voxel -> point cloud: occupied voxel centres, mapped back into the mesh's frame
idx = np.argwhere(vox > 0).astype(np.float64)
extent = mesh.extents.max()
vox_pts = (idx + 0.5) / 32 * extent - extent / 2 + mesh.bounds.mean(axis=0)
np.savetxt(os.path.join(OUT, "chair_voxel_centres.xyz"), vox_pts, fmt="%.6f")
print("voxels -> points:", vox_pts.shape)

# 4. Point cloud -> mesh. Both methods need oriented normals; we take them from the source faces, which is w
hat a
# scanner with normals would give us. Estimating them instead (estimate_normals) works but flips signs on th
in parts.
pcd = o3d.geometry.PointCloud(o3d.utility.Vector3dVector(points))
pcd.normals = o3d.utility.Vector3dVector(mesh.face_normals[face_indices])
o3d.io.write_point_cloud(os.path.join(OUT, "chair_sampled.ply"), pcd)

poisson, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd, depth=8)
densities = np.asarray(densities)
poisson.remove_vertices_by_mask(densities < np.quantile(densities, 0.05))  # trim the low-support bubble
o3d.io.write_triangle_mesh(os.path.join(OUT, "chair_poisson.ply"), poisson)
print("points -> mesh (Poisson):", len(poisson.vertices), "vertices,", len(poisson.triangles), "triangles")

spacing = np.mean(pcd.compute_nearest_neighbor_distance())
radii = o3d.utility.DoubleVector([spacing * k for k in (1.5, 3, 6)])
bpa = o3d.geometry.TriangleMesh.create_from_point_cloud_ball_pivoting(pcd, radii)
o3d.io.write_triangle_mesh(os.path.join(OUT, "chair_bpa.ply"), bpa)
print("points -> mesh (Ball Pivoting):", len(bpa.vertices), "vertices,", len(bpa.triangles), "triangles")

# Reconstruction error: symmetric Chamfer distance against fresh samples of the original surface
ref = trimesh.sample.sample_surface(mesh, 20000, seed=1)[0]


def chamfer(a, b):
    pa, pb = o3d.geometry.PointCloud(o3d.utility.Vector3dVector(a)), o3d.geometry.PointCloud(o3d.utility.Vec
tor3dVector(b))
    return 0.5 * (np.mean(pa.compute_point_cloud_distance(pb)) + np.mean(pb.compute_point_cloud_distance(pa)
))


for name, m in [("Poisson", poisson), ("Ball Pivoting", bpa)]:
    tm = trimesh.Trimesh(np.asarray(m.vertices), np.asarray(m.triangles), process=False)
    cd = chamfer(trimesh.sample.sample_surface(tm, 20000, seed=2)[0], ref)
    print("  %-13s Chamfer distance = %.4f (%.2f%% of object size)" % (name, cd, 100 * cd / extent))
print("  voxel centres  Chamfer distance = %.4f (%.2f%% of object size)" % (chamfer(vox_pts, ref), 100 * cha
mfer(vox_pts, ref) / extent))


# Figure: one panel per representation, same viewpoint
def scatter(ax, p, title, s=0.5):
    ax.scatter(p[:, 0], p[:, 2], p[:, 1], s=s, c=p[:, 1], cmap="viridis")
    ax.set_title(title, fontsize=10)


def trisurf(ax, v, f, title):
    ax.plot_trisurf(v[:, 0], v[:, 2], v[:, 1], triangles=f, color="#9ab", edgecolor="none", linewidth=0)
    ax.set_title(title, fontsize=10)


fig = plt.figure(figsize=(15, 3.4))
axes = [fig.add_subplot(1, 5, i + 1, projection="3d") for i in range(5)]
trisurf(axes[0], mesh.vertices, mesh.faces, "Original mesh\n(%d faces)" % len(mesh.faces))
scatter(axes[1], points, "Mesh -> points\n(4096 samples)")
axes[2].voxels(np.transpose(vox > 0, (0, 2, 1)), facecolors="#c84", edgecolor="#0003", linewidth=0.2)
axes[2].set_title("Mesh -> voxels\n(32$^3$, %d occupied)" % int(vox.sum()), fontsize=10)
trisurf(axes[3], np.asarray(poisson.vertices), np.asarray(poisson.triangles), "Points -> mesh\n(Poisson, dep
th 8)")
trisurf(axes[4], np.asarray(bpa.vertices), np.asarray(bpa.triangles), "Points -> mesh\n(Ball Pivoting)")
for ax in axes:
    ax.set_axis_off()
    ax.view_init(elev=20, azim=-60)
    # zoom: mplot3d pads each panel heavily, which leaves the shapes tiny in a 5-panel row
```

```
    ax.set_box_aspect(np.ptp(mesh.vertices[:, [0, 2, 1]], axis=0) if ax is not axes[2] else None, zoom=1.45)
plt.subplots_adjust(left=0, right=1, bottom=0, top=0.88, wspace=0)
plt.savefig(FIG, dpi=150, bbox_inches="tight", pad_inches=0.05)
print("wrote", os.path.relpath(FIG, os.getcwd()))
```

B.6 prep_modelnet40.py

```
"""Builds the ModelNet40 cache files that Pointnet_Pointnet2_pytorch's loader expects, straight from the zip
.

The unzipped "normal resampled" set is ~7 GB of text; this streams each shape out of the archive and writes
the
exact pickle that ModelNetDataLoader(process_data=True) would write (first 1024 of the 10k points, xyz + nor
mals),
so train_classification.py --process_data loads it without the raw .txt files being on disk.
"""
import os
import pickle
import sys
import zipfile

import numpy as np
from tqdm import tqdm

ZIP = sys.argv[1] if len(sys.argv) > 1 else "data/modelnet40_normal_resampled.zip"
OUT = "Pointnet_Pointnet2_pytorch/data/modelnet40_normal_resampled"
NPOINTS = 1024
PREFIX = "modelnet40_normal_resampled/"

os.makedirs(OUT, exist_ok=True)
zf = zipfile.ZipFile(ZIP)
for name in ["modelnet40_shape_names.txt", "modelnet40_train.txt", "modelnet40_test.txt",
             "modelnet10_shape_names.txt", "modelnet10_train.txt", "modelnet10_test.txt"]:
    with open(os.path.join(OUT, name), "wb") as f:
        f.write(zf.read(PREFIX + name))

classes = {c: i for i, c in enumerate(l.rstrip() for l in open(os.path.join(OUT, "modelnet40_shape_names.txt
"))))}
for split in ["train", "test"]:
    save_path = os.path.join(OUT, "modelnet40_%s_%dpts.dat" % (split, NPOINTS))
    if os.path.exists(save_path):
        print("exists:", save_path)
        continue
    ids = [l.rstrip() for l in open(os.path.join(OUT, "modelnet40_%s.txt" % split))]
    points, labels = [], []
    for sid in tqdm(ids, desc=split):
        cls = "_".join(sid.split("_")[:-1])
        with zf.open("%s%s/%s.txt" % (PREFIX, cls, sid)) as f:
            pts = np.loadtxt(f, delimiter=",", max_rows=NPOINTS).astype(np.float32)
        points.append(pts)
        labels.append(np.array([classes[cls]]).astype(np.int32))
    with open(save_path, "wb") as f:
        pickle.dump([points, labels], f)
    print("wrote %s (%d shapes)" % (save_path, len(ids)))
```

B.7 make_figures.py

```
"""Generates the report figures from the training logs and saved outputs, and holds the log parsers build_re
port.py uses.

Runs at any point during training: each figure is drawn from whatever epochs have been logged so far.
(figures/representations.png comes from section_7_5/convert_representations.py.)
"""
import json
import os
import re

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
FIGS = os.path.join(HERE, "figures")
PN2_LOG = os.path.join(HERE, "Pointnet_Pointnet2_pytorch", "log", "classification", "pointnet2_modelnet40",
"logs",
                       "pointnet2_cls_ssg.txt")
PN2_EVAL = os.path.join(HERE, "Pointnet_Pointnet2_pytorch", "log", "classification", "pointnet2_modelnet40",
 "eval.txt")
MESH_DIR = os.path.join(HERE, "MeshCNN", "checkpoints", "shrec16")
VOXEL_JSON = os.path.join(HERE, "outputs", "voxel_cnn_results.json")
SPHERE = os.path.join(HERE, "section_7_1", "data", "sphere.xyz")

BLUE, ORANGE, GREEN = "#2a78d6", "#eb6834", "#2f9e6e"
INK, MUTED, GRID = "#0b0b0b", "#52514e", "#e4e3df"
plt.rcParams.update({
```

```
    "font.size": 9, "axes.edgecolor": MUTED, "axes.labelcolor": INK, "xtick.color": MUTED,
    "ytick.color": MUTED, "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.6, "savefig.dpi": 200,
    "savefig.bbox": "tight", "axes.axisbelow": True, "figure.facecolor": "white",
})


# ---------------------------------------------------------------------------------------------- log parsers
def parse_pointnet2(path=PN2_LOG, eval_path=PN2_EVAL):
    """Per-epoch train/test accuracy from the repo's logger, plus the test_classification.py result if prese
nt."""
    epochs, cur, starts = [], None, []
    for line in open(path):
        if m := re.search(r"^(\S+ \S+),\d+ .*Epoch (\d+) \(", line):
            cur = {"epoch": int(m.group(2))}
            starts.append(np.datetime64(m.group(1).replace(" ", "T")))
        elif cur and (m := re.search(r"Train Instance Accuracy: ([\d.]+)", line)):
            cur["train_acc"] = float(m.group(1))
        elif cur and (m := re.search(r"Test Instance Accuracy: ([\d.]+), Class Accuracy: ([\d.]+)", line)):
            cur["test_acc"], cur["test_class_acc"] = float(m.group(1)), float(m.group(2))
            epochs.append(cur)
            cur = None
    out = {"history": epochs}
    if len(starts) > 1:  # epoch k's start-to-start time covers its training and its test pass
        out["sec_per_epoch"] = float((starts[-1] - starts[0]) / np.timedelta64(1, "s") / (len(starts) - 1))
    if epochs:
        best = max(epochs, key=lambda e: e["test_acc"])
        out.update(best_epoch=best["epoch"], best_test_acc=best["test_acc"],
                   best_class_acc=max(e["test_class_acc"] for e in epochs))
    if os.path.exists(eval_path):
        m = re.search(r"Test Instance Accuracy: ([\d.]+), Class Accuracy: ([\d.]+)", open(eval_path).read())
        if m:
            out["eval_test_acc"], out["eval_class_acc"] = float(m.group(1)), float(m.group(2))
    return out


def parse_meshcnn(run_dir=MESH_DIR):
    """Per-epoch test accuracy and mean training loss; a resumed run's later entries overwrite earlier ones.
"""
    acc = {}
    for line in open(os.path.join(run_dir, "testacc_log.txt")):
        if m := re.search(r"epoch: (\d+), TEST ACC: \[([\d.]+) %\]", line):
            acc[int(m.group(1))] = float(m.group(2)) / 100
    losses = {}
    for line in open(os.path.join(run_dir, "loss_log.txt")):
        if m := re.search(r"\(epoch: (\d+), iters: \d+.*\) loss: ([\d.]+)", line):
            losses.setdefault(int(m.group(1)), []).append(float(m.group(2)))
    ep = sorted(acc)
    hist = [{"epoch": e, "test_acc": acc[e], "loss": float(np.mean(losses[e])) if e in losses else None} for
 e in ep]
    out = {"history": hist}
    # MeshCNN logs no timestamps, but it saves <epoch>_net.pth at the end of every epoch
    saved = sorted(os.path.getmtime(p) for e in ep if os.path.exists(p := os.path.join(run_dir, "%d_net.pth"
 % e)))
    if len(saved) > 1:
        out["sec_per_epoch"] = (saved[-1] - saved[0]) / (len(saved) - 1)
    if hist:
        best = max(hist, key=lambda h: h["test_acc"])
        last10 = [h["test_acc"] for h in hist[-10:]]
        out.update(best_epoch=best["epoch"], best_test_acc=best["test_acc"], last_epoch=ep[-1],
                   last10_mean=float(np.mean(last10)), last10_std=float(np.std(last10)))
    return out


def load_voxel(path=VOXEL_JSON):
    return json.load(open(path)) if os.path.exists(path) else None


# ---------------------------------------------------------------------------------------------- figures
def fig_sphere():
    pts = np.loadtxt(SPHERE)
    r = np.linalg.norm(pts, axis=1)
    fig = plt.figure(figsize=(8.4, 3.4))
    ax = fig.add_subplot(1, 2, 1, projection="3d")
    ax.scatter(*pts.T, s=0.6, c=pts[:, 2], cmap="viridis")
    ax.set_box_aspect((1, 1, 1))
    ax.set_axis_off()
    ax.set_title("3000 points on a noisy unit sphere", fontsize=9)
    ax = fig.add_subplot(1, 2, 2)
    ax.hist(r, bins=50, color=BLUE, alpha=0.85)
    ax.axvline(1.0, color=ORANGE, lw=1.2, ls="--", label="unit radius")
    ax.set(xlabel="distance from origin", ylabel="points",
           title="radius: mean %.4f, std %.4f (noise $\\sigma$ = 0.01)" % (r.mean(), r.std()))
    ax.legend(frameon=False)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGS, "fig_sphere.png"))
    plt.close()
```

```python


def fig_pointnet2(pn):
    h = pn["history"]
    ep = [e["epoch"] for e in h]
    fig, ax = plt.subplots(figsize=(6.6, 3.3))
    ax.plot(ep, [e["train_acc"] for e in h], "o-", ms=3, color=MUTED, label="train (instance)")
    ax.plot(ep, [e["test_acc"] for e in h], "o-", ms=3, color=BLUE, label="test (instance)")
    ax.plot(ep, [e["test_class_acc"] for e in h], "o--", ms=3, color=ORANGE, label="test (class average)")
    ax.set(xlabel="epoch", ylabel="accuracy", title="PointNet++ (SSG) on ModelNet40", ylim=(0.4, 1.0))
    ax.set_xticks(ep if len(ep) <= 25 else ep[::2])
    ax.legend(frameon=False, loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGS, "fig_pointnet2.png"))
    plt.close()


def fig_meshcnn(mc):
    h = mc["history"]
    ep = np.array([e["epoch"] for e in h])
    acc = np.array([e["test_acc"] for e in h])
    k = 5
    smooth = np.convolve(acc, np.ones(k) / k, mode="valid")
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.2, 3.2))
    a1.plot(ep, [e["loss"] for e in h], color=MUTED, lw=1.2)
    a1.set(xlabel="epoch", ylabel="mean training loss", title="MeshCNN on SHREC'16: loss")
    a2.plot(ep, acc, "o", ms=2.5, color=BLUE, alpha=0.45, label="test accuracy (per epoch)")
    a2.plot(ep[k - 1:], smooth, color=BLUE, lw=1.6, label="%d-epoch moving average" % k)
    a2.set(xlabel="epoch", ylabel="accuracy", ylim=(0, 1), title="MeshCNN on SHREC'16: test accuracy")
    a2.legend(frameon=False, loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGS, "fig_meshcnn.png"))
    plt.close()


def fig_voxel(vx):
    h = vx["history"]
    ep = [e["epoch"] for e in h]
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.2, 3.2))
    a1.plot(ep, [e["loss"] for e in h], "o-", ms=3, color=MUTED)
    a1.set(xlabel="epoch", ylabel="training loss", title="VoxelCNN on ModelNet10: loss")
    a2.plot(ep, [e["train_acc"] for e in h], "o-", ms=3, color=MUTED, label="train")
    a2.plot(ep, [e["test_acc"] for e in h], "o-", ms=3, color=BLUE, label="test (instance)")
    a2.plot(ep, [e["test_class_acc"] for e in h], "o--", ms=3, color=ORANGE, label="test (class average)")
    a2.set(xlabel="epoch", ylabel="accuracy", ylim=(0.5, 1.0), title="VoxelCNN on ModelNet10: accuracy")
    a2.legend(frameon=False, loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGS, "fig_voxel_curves.png"))
    plt.close()

    classes = vx["classes"]
    conf = np.array(vx["best"]["confusion"])
    norm = conf / conf.sum(1, keepdims=True)
    fig, ax = plt.subplots(figsize=(5.6, 4.8))
    ax.grid(False)
    ax.imshow(norm, cmap="Blues", vmin=0, vmax=1)
    for r in range(len(classes)):
        for c in range(len(classes)):
            if conf[r, c]:
                ax.text(c, r, conf[r, c], ha="center", va="center", fontsize=7.5,
                        color="white" if norm[r, c] > 0.5 else INK)
    ax.set_xticks(range(len(classes)), classes, rotation=45, ha="right")
    ax.set_yticks(range(len(classes)), classes)
    ax.set(xlabel="predicted class", ylabel="true class")
    plt.tight_layout()
    plt.savefig(os.path.join(FIGS, "fig_voxel_confusion.png"))
    plt.close()


if __name__ == "__main__":
    os.makedirs(FIGS, exist_ok=True)
    fig_sphere()
    pn, mc, vx = parse_pointnet2(), parse_meshcnn(), load_voxel()
    fig_pointnet2(pn)
    fig_meshcnn(mc)
    print("PointNet++: %d epochs logged, best test %.3f" % (len(pn["history"]), pn.get("best_test_acc", float("nan"))))
    print("MeshCNN:    %d epochs logged, best test %.3f" % (len(mc["history"]), mc.get("best_test_acc", float("nan"))))
    if vx:
        fig_voxel(vx)
        print("VoxelCNN:   %d epochs, best test %.3f" % (len(vx["history"]), vx["best"]["test_acc"]))
    else:
        print("VoxelCNN:   no results yet")
    print("figures in", os.path.relpath(FIGS))
```

B.8 Patch to Pointnet_Pointnet2_pytorch (CUDA to device-agnostic)

```
diff --git a/test_classification.py b/test_classification.py
index 79ac097..e103f1f 100644
--- a/test_classification.py
+++ b/test_classification.py
@@ -3,6 +3,7 @@ Author: Benny
 Date: Nov 2019
 """
 from data_utils.ModelNetDataLoader import ModelNetDataLoader
+
 import argparse
 import numpy as np
 import os
@@ -12,6 +13,9 @@ from tqdm import tqdm
 import sys
 import importlib

+# Original code is CUDA-only; fall back to Apple-silicon MPS, then CPU
+DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else...
+
 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
 ROOT_DIR = BASE_DIR
 sys.path.append(os.path.join(ROOT_DIR, 'models'))
@@ -39,10 +43,10 @@ def test(model, loader, num_class=40, vote_num=1):

     for j, (points, target) in tqdm(enumerate(loader), total=len(loader)):
         if not args.use_cpu:
-            points, target = points.cuda(), target.cuda()
+            points, target = points.to(DEVICE), target.to(DEVICE)

         points = points.transpose(2, 1)
-        vote_pool = torch.zeros(target.size()[0], num_class).cuda()
+        vote_pool = torch.zeros(target.size()[0], num_class).to(DEVICE)

         for _ in range(vote_num):
             pred, _ = classifier(points)
@@ -90,7 +94,8 @@ def main(args):
     log_string('Load dataset ...')
     data_path = 'data/modelnet40_normal_resampled/'

-    test_dataset = ModelNetDataLoader(root=data_path, args=args, split='test', process_data=False)
+    # read the same 1024-point cache training used (the raw .txt files are not kept)
+    test_dataset = ModelNetDataLoader(root=data_path, args=args, split='test', process_data=True)
     testDataLoader = torch.utils.data.DataLoader(test_dataset, batch_size=args.batch_size, shuffle=False, num_...

     '''MODEL LOADING'''
@@ -100,9 +105,9 @@ def main(args):

     classifier = model.get_model(num_class, normal_channel=args.use_normals)
     if not args.use_cpu:
-        classifier = classifier.cuda()
+        classifier = classifier.to(DEVICE)

-    checkpoint = torch.load(str(experiment_dir) + '/checkpoints/best_model.pth')
+    checkpoint = torch.load(str(experiment_dir) + '/checkpoints/best_model.pth', map_location=DEVICE, weights_...
     classifier.load_state_dict(checkpoint['model_state_dict'])

     with torch.no_grad():
diff --git a/train_classification.py b/train_classification.py
index 229f49d..9020b98 100644
--- a/train_classification.py
+++ b/train_classification.py
@@ -19,6 +19,9 @@ from pathlib import Path
 from tqdm import tqdm
 from data_utils.ModelNetDataLoader import ModelNetDataLoader

+# Original code is CUDA-only; fall back to Apple-silicon MPS, then CPU
+DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else...
+
 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
 ROOT_DIR = BASE_DIR
 sys.path.append(os.path.join(ROOT_DIR, 'models'))
@@ -57,7 +60,7 @@ def test(model, loader, num_class=40):
     for j, (points, target) in tqdm(enumerate(loader), total=len(loader)):

         if not args.use_cpu:
-            points, target = points.cuda(), target.cuda()
+            points, target = points.to(DEVICE), target.to(DEVICE)

         points = points.transpose(2, 1)
         pred, _ = classifier(points)
@@ -135,11 +138,11 @@ def main(args):
     classifier.apply(inplace_relu)

     if not args.use_cpu:
-        classifier = classifier.cuda()
+        classifier = classifier.to(DEVICE)
```

```
-        criterion = criterion.cuda()
+        classifier = classifier.to(DEVICE)
+        criterion = criterion.to(DEVICE)

     try:
-        checkpoint = torch.load(str(exp_dir) + '/checkpoints/best_model.pth')
+        checkpoint = torch.load(str(exp_dir) + '/checkpoints/best_model.pth', map_location=DEVICE, weights_onl...
         start_epoch = checkpoint['epoch']
         classifier.load_state_dict(checkpoint['model_state_dict'])
         log_string('Use pretrain model')
@@ -183,7 +186,7 @@ def main(args):
             points = points.transpose(2, 1)

             if not args.use_cpu:
-                points, target = points.cuda(), target.cuda()
+                points, target = points.to(DEVICE), target.to(DEVICE)

             pred, trans_feat = classifier(points)
             loss = criterion(pred, target.long(), trans_feat)
```

B.9 Patch to MeshCNN (NumPy 1.24+ compatibility)

```
diff --git a/models/layers/mesh_prepare.py b/models/layers/mesh_prepare.py
index 47e827c..ba0af3f 100644
--- a/models/layers/mesh_prepare.py
+++ b/models/layers/mesh_prepare.py
@@ -10,7 +10,7 @@ def fill_mesh(mesh2fill, file: str, opt):
     else:
         mesh_data = from_scratch(file, opt)
         np.savez_compressed(load_path, gemm_edges=mesh_data.gemm_edges, vs=mesh_data.vs, edges=mesh_data.edges,
-                            edges_count=mesh_data.edges_count, ve=mesh_data.ve, v_mask=mesh_data.v_mask,
+                            edges_count=mesh_data.edges_count, ve=_ragged(mesh_data.ve), v_mask=mesh_data.v_mask,
                             filename=mesh_data.filename, sides=mesh_data.sides,
                             edge_lengths=mesh_data.edge_lengths, edge_areas=mesh_data.edge_areas,
                             features=mesh_data.features)
@@ -26,6 +26,12 @@ def fill_mesh(mesh2fill, file: str, opt):
     mesh2fill.features = mesh_data['features']
     mesh2fill.sides = mesh_data['sides']

+def _ragged(lists):
+    # NumPy >= 1.24 refuses to build ragged arrays implicitly; store as a 1-D object array
+    out = np.empty(len(lists), dtype=object)
+    out[:] = [list(l) for l in lists]
+    return out
+
 def get_mesh_path(file: str, num_aug: int):
     filename, _ = os.path.splitext(file)
     dir_name = os.path.dirname(filename)
```